

Exact Two-Source Categorical SxPID Count-to-Atom Bridge

A bounded 24-coordinate Lean formalization of components, Möbius atoms, exact count products, and sign reduction

pid-rs project analysis

10 August 2026

Abstract

This report documents the bounded Lean formalization for the complete two-source finite categorical shared-exclusions surface: four cumulative nodes and four Möbius atoms, each carrying informative, misinformative, and signed-net components. The checked module fixes an explicit 24-coordinate order, proves the concrete two-source Möbius and zeta transforms are inverse, proves source-labelled coordinate-swap equivariance, and proves that finite empirical averaging commutes with the transform. Starting from an arbitrary supplied natural-valued count function with positive total, it relates every supported local component to an exact rational logarithm argument. Every averaged coordinate is then represented as $N^{-1} \log Q$ for a positive exact rational product Q , so positive, negative, and zero values reduce to comparison of Q with one.

The categorical event functional, component split, and Möbius atom construction are defined in the Makkeh–Gutknecht–Wibral publications. The Lean encodings, coordinate order, supplied-count interface, exact product layer, theorem factoring, and assurance packet are project-defined formalization work; they define no new PID and carry no scientific-priority claim. Exact pinned kernel replay, 80 registered static and baseline-first isolated changes, categorical-only scope binding, deterministic PDF checks, page inspection, and separate read-only repository-workflow reviews are recorded. Component-atom nonnegativity, Rust and binary64 refinement, standalone-certifier refinement, statistical conclusions, and higher-source lattices are explicit nonclaims.

Contents

1	Status and decision boundary	2
2	Scientific provenance and project contribution	3
2.1	Paper-defined object	3
2.2	Project-defined formalization	3
3	Inherited supplied-count event model	4
3.1	Complete keys and positive support	4
3.2	Keyed events	4
4	Fixed cumulative nodes and components	4
4.1	Four cumulative nodes	4
4.2	Three local component arguments	5
4.3	Averaged cumulatives	5

5	Concrete two-source Möbius algebra	5
5.1	Coefficient tables	5
5.2	Integer-row representation	6
5.3	Source-labelled coordinate exchange	6
6	Averaging and component linearity	6
6.1	Pointwise atoms	6
6.2	Net component	7
7	The exact 24-coordinate surface	7
7.1	Types and order	7
7.2	Uniform count-expression theorem	7
8	Exact rational products	8
8.1	Cumulative products	8
8.2	Atom products	8
8.3	Uniform coordinate normalization	8
9	Exact sign and zero reduction	9
9.1	Component-nonnegativity is not proved	9
10	Theorem crosswalk	9
11	Twenty-four-lens bounded review	10
12	Residual boundaries	11
13	Acceptance evidence	12
14	Conclusion	12

1 Status and decision boundary

Bounded accepted status. The exact source is accepted only for the supplied-count, fixed-two-source categorical formal scope. It is not a publication-correspondence, component-sign, executable-refinement, higher-source, or population theorem.

The checked source is `TwoSourceMöbiusAtomBridge.lean` under `audit/formal/lean/PidFiniteConvergence`. It imports the completed supplied-count event bridge and adds the concrete two-source component/atom layer. The accepted target is exactly

$$3 \times (4 + 4) = 24. \quad (1)$$

The three components are informative, misinformative, and signed net. The four cumulative nodes are source one, source two, joint sources, and redundancy. The four atoms are unique one, unique two, synergy, and redundancy. Equation (1) is a finite categorical surface, not a continuous estimator and not a higher-source lattice.

The acceptance evidence comprises the following bounded layers:

1. pinned Lean toolchain, dependency, source, declaration, and axiom-basis replay;
2. meaningful negative semantic variations of order, events, coefficients, components, support, weights, products, comparisons, and scope prose;
3. method-catalog, assurance-registry, formal-map, limitations, and changelog integration;

4. deterministic PDF generation, extraction, font and geometry checks, and page inspection; and
5. separate read-only mathematical, provenance, formal, implementation-boundary, and presentation reviews within the repository workflow.

2 Scientific provenance and project contribution

2.1 Paper-defined object

Makkeh, Gutknecht, and Wibrat define the finite categorical pointwise shared-exclusions construction, its informative and misinformative decomposition, the signed-net quantity, and its partial-information atoms through Möbius inversion [1]. Gutknecht, Wibrat, and Makkeh develop the associated part-whole and formal-logic organization [2].

The following scientific content is therefore paper-defined:

- keyed source-event disjunctions and keyed target restrictions;
- pointwise informative and misinformative cumulative components;
- signed net as informative minus misinformative;
- the passage from cumulatives to atoms by Möbius inversion; and
- empirical averaging of pointwise quantities.

2.2 Project-defined formalization

The repository contributes a formal and assurance realization of that fixed object:

- heterogeneous finite source types indexed by $\text{Fin } 2$ and a finite target type;
- explicit Lean constructors for components, nodes, atoms, and cumulative-or-atom coordinates;
- fixed node, atom, component, and complete 24-coordinate orders;
- concrete integer Möbius and zeta coefficient tables;
- exact supplied-count component arguments on positive support;
- exact rational and real product representatives for every coordinate;
- generic additive-group inverse and reconstruction theorems;
- theorem-level links between pointwise averaging, cumulative inversion, count formulas, products, and signs; and
- a bounded replay and review plan.

These are project-defined encodings, proofs, and engineering boundaries. They do not define a new PID measure. The correspondence from publication equations to the selected Lean definitions is a human-reviewed transcription boundary, not a theorem that Lean derives from publication text.

3 Inherited supplied-count event model

3.1 Complete keys and positive support

Let the two source alphabets $\mathcal{S}_1, \mathcal{S}_2$ and target alphabet $\mathcal{T}$ be finite. A complete key is

$$z = (s_1, s_2, t) \in \mathcal{S}_1 \times \mathcal{S}_2 \times \mathcal{T}.$$

Supply an arbitrary natural count function c and define

$$N = \sum_z c(z) > 0, \quad Z_+ = \{z : c(z) > 0\}. \quad (2)$$

Zero-count keys are permitted. The exact empirical law is

$$\hat{p}_c(z) = \frac{c(z)}{N}, \quad (3)$$

and all local logarithms and empirical sums below range over Z_+ .

3.2 Keyed events

For a cumulative node α and supported anchor z , let

$$E_\alpha(z) = \text{the keyed source event}, \quad (4)$$

$$T(z) = \text{the keyed target branch}, \quad (5)$$

$$E_{\alpha,t}(z) = E_\alpha(z) \cap T(z). \quad (6)$$

Write their exact natural counts as

$$C_\alpha(z) = \text{Count}(E_\alpha(z)), \quad C_t(z) = \text{Count}(T(z)), \quad C_{\alpha,t}(z) = \text{Count}(E_{\alpha,t}(z)). \quad (7)$$

The completed count/event dependency proves that a supported anchor belongs to each defining event. Consequently

$$C_\alpha(z) > 0, \quad C_t(z) > 0, \quad C_{\alpha,t}(z) > 0 \quad (z \in Z_+). \quad (8)$$

Positivity in equation (8) is derived from anchor membership; it is not an extra statistical assumption.

4 Fixed cumulative nodes and components

4.1 Four cumulative nodes

Table 1: Exact two-source cumulative-node order.

Position	Lean node	Keyed source-event meaning	Symbol
1	<code>sourceOne</code>	Source one matches its anchor value.	C_1
2	<code>sourceTwo</code>	Source two matches its anchor value.	C_2
3	<code>jointSources</code>	Both sources match their anchor values.	C_{12}
4	<code>redundancy</code>	Source one or source two matches its anchor value.	C_R

The redundancy event in Table 1 is a union; the joint-source event is the intersection of the two singleton source branches. Cumulative source-one and source-two values are not unique atoms, and the joint-source cumulative is not the synergy atom.

4.2 Three local component arguments

For $u \in \{+, -, n\}$, define the exact rational local arguments

$$q_\alpha^+(z) = \frac{N}{C_\alpha(z)}, \quad (9)$$

$$q_\alpha^-(z) = \frac{C_t(z)}{C_{\alpha,t}(z)}, \quad (10)$$

$$q_\alpha^n(z) = \frac{NC_{\alpha,t}(z)}{C_\alpha(z)C_t(z)}. \quad (11)$$

Equation (8) makes every argument strictly positive. Direct rational algebra gives

$$q_\alpha^n(z) = \frac{q_\alpha^+(z)}{q_\alpha^-(z)}. \quad (12)$$

The local component values in nats are

$$i_\alpha^u(z) = \log q_\alpha^u(z). \quad (13)$$

The logarithm quotient rule and equation (12) give

$$i_\alpha^n(z) = i_\alpha^+(z) - i_\alpha^-(z). \quad (14)$$

The checked module proves the informative and misinformative empirical probability expressions equal the logarithms in equations (9) and (10); the inherited module already supplies the net case. A single component selector then states the result uniformly.

4.3 Averaged cumulatives

For each component and node, define

$$C_\alpha^u = \sum_{z \in Z_+} \frac{c(z)}{N} i_\alpha^u(z). \quad (15)$$

Finite-sum linearity yields

$$C_\alpha^n = C_\alpha^+ - C_\alpha^-. \quad (16)$$

Equation (15) uses exact count weights. It inserts no pseudo-count, smoothing rule, uniform-on-support weight, clipping, or absolute value.

5 Concrete two-source Möbius algebra

5.1 Coefficient tables

Use cumulative column order $[C_1, C_2, C_{12}, C_R]$ and atom row order $[U_1, U_2, S, R]$. The checked module fixes

$$M = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ -1 & -1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (17)$$

The concrete Möbius transform is therefore

$$\Pi_R^u = C_R^u, \quad (18)$$

$$\Pi_{U_1}^u = C_1^u - C_R^u, \quad (19)$$

$$\Pi_{U_2}^u = C_2^u - C_R^u, \quad (20)$$

$$\Pi_S^u = C_{12}^u - C_1^u - C_2^u + C_R^u. \quad (21)$$

The inverse zeta coefficient matrix is

$$Z = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (22)$$

with cumulative row order and atom column order as above.

Proposition 5.1 (Inverse algebra). *For any additive commutative group, the concrete transforms satisfy*

$$\text{Zeta}(\text{Mob}(C)) = C, \quad \text{Mob}(\text{Zeta}(\Pi)) = \Pi.$$

The source proves Proposition 5.1 by cases on the four positions and commutative group algebra. The theorem is independent of real numbers, logarithms, and count semantics.

Corollary 5.2 (Reconstruction identities). *For each component,*

$$C_R^u = \Pi_R^u, \quad (23)$$

$$C_1^u = \Pi_{U_1}^u + \Pi_R^u, \quad (24)$$

$$C_2^u = \Pi_{U_2}^u + \Pi_R^u, \quad (25)$$

$$C_{12}^u = \Pi_{U_1}^u + \Pi_{U_2}^u + \Pi_S^u + \Pi_R^u. \quad (26)$$

5.2 Integer-row representation

The checked module separately states that each concrete transform equals the finite sum of its integer coefficient row. This matters because the coefficient table, rather than only a simplified formula, is part of the intended formal contract. It also proves that every nonredundancy Möbius row sums to zero while the redundancy row sums to one.

5.3 Source-labelled coordinate exchange

Define coordinate exchange on source-labelled cumulatives and atoms by

$$C_1 \leftrightarrow C_2, \quad U_1 \leftrightarrow U_2, \quad (27)$$

while joint sources, redundancy, synergy, and redundancy atom remain fixed. The checked module proves both maps are involutions and

$$\text{Mob}(C \circ \text{swap})(\gamma) = \text{Mob}(C)(\text{swap}(\gamma)). \quad (28)$$

This is formula equivariance for an arbitrary four-entry cumulative function. It does not transport heterogeneous source types, keys, counts, laws, or inherited events, and it does not relate the order to Rust fields or a serialized record.

6 Averaging and component linearity

6.1 Pointwise atoms

Let $m_{\gamma\alpha}$ be the integer coefficient in row γ of equation (17). Define a pointwise atom by

$$\pi_\gamma^u(z) = \sum_{\alpha} m_{\gamma\alpha} i_\alpha^u(z). \quad (29)$$

Its positive-support empirical average is

$$\bar{\pi}_\gamma^u = \sum_{z \in Z_+} \frac{c(z)}{N} \pi_\gamma^u(z). \quad (30)$$

Proposition 6.1 (Averaging/Möbius commutation). *For every component and atom,*

$$\bar{\pi}_\gamma^u = \sum_\alpha m_{\gamma\alpha} C_\alpha^u = \Pi_\gamma^u. \quad (31)$$

The proof expands the four atom cases and distributes multiplication and finite sums over addition and subtraction. It establishes equality of “invert each supported point then average” and “average each cumulative then invert” for the selected finite empirical formalization.

6.2 Net component

The Möbius transform is additive and commutes with subtraction. Combining this fact with equation (16) yields

$$\Pi_\gamma^n = \Pi_\gamma^+ - \Pi_\gamma^-. \quad (32)$$

Equation (32) is an exact signed identity. No intermediate term or final atom is silently clamped.

7 The exact 24-coordinate surface

7.1 Types and order

The checked module defines three finite types:

- component: informative, misinformative, net;
- atom: unique one, unique two, synergy, redundancy; and
- coordinate: a component-indexed cumulative node or a component-indexed atom.

It fixes the explicit order in Table 2.

Table 2: Project-defined order over the 24 paper-defined quantities.

Block	Ordered coordinates
Informative cumulatives	$C_1^+, C_2^+, C_{12}^+, C_R^+$
Misinformative cumulatives	$C_1^-, C_2^-, C_{12}^-, C_R^-$
Signed-net cumulatives	$C_1^n, C_2^n, C_{12}^n, C_R^n$
Informative atoms	$\Pi_{U_1}^+, \Pi_{U_2}^+, \Pi_S^+, \Pi_R^+$
Misinformative atoms	$\Pi_{U_1}^-, \Pi_{U_2}^-, \Pi_S^-, \Pi_R^-$
Signed-net atoms	$\Pi_{U_1}^n, \Pi_{U_2}^n, \Pi_S^n, \Pi_R^n$

The source states that the coordinate type has cardinality 24, the ordered list has length 24 with no duplicate, and the list’s finite-set projection equals the complete coordinate universe. This fixes a mathematical order. It does not prove equality to a Rust array, report field sequence, or external encoding.

7.2 Uniform count-expression theorem

Define a count expression for a cumulative by inserting equations (9)–(11) into equation (15). Define the atom count expression by applying the same concrete Möbius transform. The checked module’s central theorem is quantified over the coordinate type:

```
theorem all_24_averaged_coordinates_empirical_eq_count_expression
  (count : CategoricalKey (Fin 2) sourceValue targetValue -> Nat)
  (coordinate : SxPid2Coordinate)
```

```
(h_total : 0 < totalCount count) :
  averagedSxPid2Coordinate (empiricalLaw count) coordinate =
    sxPid2CountCoordinateExpression count coordinate
```

The cumulative branch uses the supported local empirical-log theorem. The atom branch transports the four cumulative equalities through the Möbius transform.

8 Exact rational products

8.1 Cumulative products

For each component and cumulative node, define

$$P_\alpha^u = \prod_{z \in Z_+} (q_\alpha^u(z))^{c(z)}. \quad (33)$$

Every factor is a positive rational, and each exponent is a natural count. The checked module defines both an exact rational value and its real counterpart, and proves that casting the exact rational product gives the real product.

Proposition 8.1 (Cumulative normalization). *For every component and cumulative node,*

$$C_\alpha^u = \frac{1}{N} \log P_\alpha^u. \quad (34)$$

The proof applies the logarithm of a finite product and the logarithm of a natural power, then factors the positive scalar $1/N$ out of the weighted sum.

8.2 Atom products

Write A_γ^u for an atom product and P_α^u for a cumulative product. The multiplicative form of equations (18)–(21) is

$$A_{U_1}^u = \frac{P_1^u}{P_R^u}, \quad (35)$$

$$A_{U_2}^u = \frac{P_2^u}{P_R^u}, \quad (36)$$

$$A_S^u = \frac{P_{12}^u P_R^u}{P_1^u P_2^u}, \quad (37)$$

$$A_R^u = P_R^u. \quad (38)$$

All cumulative products are positive, so every quotient is defined and positive.

Proposition 8.2 (Atom normalization). *For every component and atom,*

$$\Pi_\gamma^u = \frac{1}{N} \log A_\gamma^u. \quad (39)$$

The proof uses equations (34) and (35)–(38), together with logarithm identities justified by strict positivity.

8.3 Uniform coordinate normalization

The cumulative and atom results combine into one statement:

$$V(c, \xi) = \frac{1}{N} \log Q(c, \xi) \quad \text{for every one of the 24 coordinates } \xi. \quad (40)$$

The checked module also proves

$$Q_{\mathbb{R}}(c, \xi) = (Q_{\mathbb{Q}}(c, \xi) : \mathbb{R}), \quad (41)$$

where $Q_{\mathbb{Q}}$ is an exact rational value.

9 Exact sign and zero reduction

Equation (2) gives $1/N > 0$, and product positivity gives $Q(c, \xi) > 0$. The strict monotonicity of the natural logarithm on positive reals and $\log 1 = 0$ yield the following.

Theorem 9.1 (All-coordinate sign reduction). *For every supplied count function with positive total and every coordinate ξ ,*

$$V(c, \xi) > 0 \iff Q(c, \xi) > 1, \quad (42)$$

$$V(c, \xi) < 0 \iff Q(c, \xi) < 1, \quad (43)$$

$$V(c, \xi) = 0 \iff Q(c, \xi) = 1. \quad (44)$$

Theorem 9.1 removes transcendental approximation from the mathematical classification once the exact rational product is available. It does not prove that any current parser, standalone certifier, Rust implementation, or report produces the same rational product or performs the comparison with a declared resource policy.

9.1 Component-nonnegativity is not proved

Equations (42)–(44) are conditional equivalences. They tell us how a coordinate’s sign follows from the corresponding product. They do not establish where the product for every informative or misinformative atom must lie. In particular, the checked module does not prove

$$\Pi_{\gamma}^{+} \geq 0 \quad \text{or} \quad \Pi_{\gamma}^{-} \geq 0 \quad \text{for all } \gamma.$$

Nor does it produce a counterexample to either statement. The publication’s component-sign results must not be reported as Lean-proved by this packet unless a separate formal theorem is added and accepted.

10 Theorem crosswalk

Table 3: Main bound declaration groups.

Layer	Bound declarations
Types and order	<code>SxPid2Component</code> , <code>SxPid2Atom</code> , <code>SxPid2Coordinate</code> , the four order definitions, and their length, no-duplicate, and completeness/cardinality theorems.
Transforms	<code>sxPid2MobiusCoefficient</code> , <code>sxPid2ZetaCoefficient</code> , <code>sxPid2MobiusTransform</code> , <code>sxPid2ZetaTransform</code> , and their integer-row forms.
Inverse algebra	<code>sx_pid2_zeta_after_mobius</code> , <code>sx_pid2_mobius_after_zeta</code> , and cumulative reconstruction theorems.
Source exchange	Node and atom swap definitions, involution theorems, and <code>sx_pid2_mobius_coordinate_swap_equivariant</code> .
Components	Local, averaged-cumulative, local-atom, averaged-pointwise-atom, and averaged-atom component selectors, plus the net-subtraction theorems.
Averaging	<code>averaged_pointwise_atom_eq_mobius_of_averaged_cumulatives</code> .

Layer	Bound declarations
Local counts	Informative, misinformative, and selected component arguments; positivity; net ratio; and local empirical-log equalities.
All-coordinate counts	Cumulative, atom, and coordinate count expressions, culminating in <code>all_24_averaged_coordinates_empirical_eq_count_expression</code> .
Products	Cumulative, atom, and coordinate real/rational products, positivity, and exact-rational cast equalities.
Scaled logarithms	Cumulative and atom normalization, then <code>all_24_averaged_coordinates_eq_scaled_log_product</code> .
Signs	The all-coordinate positive, negative, and zero equivalences.

This table is a human crosswalk, not the enforcement route. The checker binds exact source bytes, 339 declarations, 246 named source theorem axiom bases, and both semantic contracts.

11 Twenty-four-lens bounded review

Table 4: Structured review of the accepted boundary.

No.	Lens	Bounded finding
1	Measure identity	The definitions remain finite categorical shared exclusions. No continuous estimator, $I_{\min}$, or other PID is substituted.
2	Publication provenance	Event, component, and Möbius mathematics is paper-defined; the Lean realization and assurance packet are project-defined.
3	Source arity	Every new scientific coordinate is fixed to two sources through <code>Fin, 2</code> .
4	Alphabet generality	The two finite source alphabets may be heterogeneous; the target is finite.
5	Count quantifier	Counts are arbitrary natural values and need not be positive at every complete key.
6	Total and support	Positive total is explicit; logarithms and averages range only over positive-count anchors.
7	Event dependency	The module imports the completed keyed event semantics rather than silently redefining source or target events.
8	Component relation	Signed net is informative minus misinformative, with no clamping or absolute value.
9	Node order	Source one, source two, joint, and redundancy have an explicit order and distinct constructors.
10	Atom order	Unique one, unique two, synergy, and redundancy are explicit and are not confused with cumulative nodes.
11	Coefficient visibility	Integer Möbius and zeta tables are definitions rather than unstated algebra.
12	Two-sided inverse	Both transform compositions are theorem targets over an additive commutative group.
13	Reconstruction	Marginal and joint cumulative reconstruction equations are explicit.

No.	Lens	Bounded finding
14	Coordinate symmetry	Swapping source-labelled coordinate positions exchanges only the source-specific node and atom pairs; no typed source-data transport is claimed.
15	Component linearity	Möbius inversion commutes with informative-minus-misinformative subtraction.
16	Averaging order	Pointwise inversion then averaging equals averaging cumulatives then inverting.
17	Coordinate completeness	Coordinate cardinality, explicit-list length, no-duplicate, and toFinset = univ facts all establish the same 24-coordinate universe.
18	Local exactness	Each supported local component is related to an exact rational count argument.
19	Empirical weighting	Every local logarithm is weighted by its exact multiplicity divided by total count.
20	Product construction	Multiplicities become natural exponents; negative Möbius coefficients become exact quotients.
21	Exact-real bridge	Real products are tied to casts of exact rational products.
22	Sign logic	Strict positivity, strict negativity, and exact zero use comparison with one and the positive scale $1/N$.
23	Refinement boundary	Rust, binary64, parsing, standalone-certifier, Python, and resource semantics remain outside the theorem.
24	Acceptance evidence	Exact kernel/source binding, 80 registered changes, categorical-only scope integration, deterministic PDF checks, page inspection, and separate read-only repository-workflow reviews are recorded.

The lenses in Table 4 are one structured review, not 24 independent proofs or 24 independent implementations.

12 Residual boundaries

The accepted bounded claim must not expand to any of the following:

1. derivation of Lean definitions from publication text;
2. row, byte, file, JSON, or other representation to the supplied count function;
3. Rust sorting, histogram extraction, or event scanning;
4. Rust NODES2, invert2, pointwise accumulation, atom fields, or order;
5. binary64, MPFR, Python, compiler, runtime, or hardware equality;
6. standalone-certifier schemas, parsers, product evaluation, comparisons, or reports;
7. integer overflow, expression normalization, allocation, cancellation, time, memory, or another resource theorem;
8. informative or misinformative atom nonnegativity;
9. sampling uncertainty, concentration, consistency, population support, or calibration;

10. continuous Ehrlich/KSG shared exclusions, fitted quantized wrappers, $I_{\min}$, or Shannon-invariant families;
11. three-source or general higher-source lattices;
12. scientific priority, uniqueness, application validity, release readiness, or downstream authority.

Executable agreement on bounded fixtures can be useful corroboration after it is explicitly performed. It does not replace a missing formal refinement theorem. Likewise, exact finite-table algebra does not imply a sampling or population conclusion.

13 Acceptance evidence

The pinned checker binds 11 Lean sources, eight imported modules, 339 declarations, and 246 named source theorem axiom bases. The original 16-example and atom 11-example semantic contracts compile with `lean -t 0`; the atom contract also checks the permitted basis of six private helper theorems. Under normal and optimized Python, the self-test rejects 49 static, five count/event, 17 atom-module, and nine atom-contract changes. The six previously missing minimum classes cover same-name inverse weakening, coordinate omission and duplication, net addition, atom-product quotient-to-multiplication, product-comparison reversal, and removal of a logarithm-product positivity premise.

The method catalog and generated assurance registry route the atom-exclusive evidence only to the stable categorical SxPID family. The Markdown/LaTeX/PDF pair is checked for deterministic rendering, strict LaTeX diagnostics, required boundary text, A4 geometry, and embedded Unicode-mapped fonts. Every rendered page was inspected for clipping, overlap, table continuation, and legibility. Separate read-only formal, integration, and document reviews within the repository workflow checked the mathematics, provenance, executable boundary, checker sensitivity, and presentation. These are not external review or independent authorship. The exact current hashes, commands, and outcomes are recorded in the versioned Lean 4.33 replay receipt. The dated phase-A receipt remains the immutable historical Lean 4.32 observation of the earlier 71-route suite; it is not evidence for the current 80-route replay. Both receipts and the residual cut are linked from the canonical Markdown audit.

14 Conclusion

The checked module closes a precise mathematical gap between the completed two-source count/event semantics and a complete finite categorical component/atom surface. It makes the two-source Möbius algebra explicit, proves that empirical averaging and inversion commute, and assigns an exact supplied-count and product representation to every one of 24 coordinates. The exact sign reduction is strong but narrow: it classifies a fixed supplied-count coordinate once its exact product is known.

The boundary is equally important. The checked module does not prove component-atom nonnegativity, does not refine Rust or another executable, does not verify binary64 arithmetic or resource behavior, and does not make a statistical or higher-source claim. This document is a bounded formal audit, not an end-to-end completion certificate for categorical SxPID software or science.

References

- [1] A. Makkeh, A. J. Gutknecht, and M. Wibral. Introducing a differentiable measure of pointwise shared information. *Physical Review E*, 103:032149, 2021. <https://doi.org/10.1103/PhysRevE.103.032149>.
- [2] A. J. Gutknecht, M. Wibral, and A. Makkeh. Bits and pieces: Understanding information decomposition from part-whole relationships and formal logic. *Proceedings of the Royal Society A*, 477, 2021. <https://doi.org/10.1098/rspa.2021.0110>.